Lo que queda por hacer

LudusERP · plan de los dos últimos sprints, por orden de prioridad

Fecha: 7 de noviembre de 2026 **Estado del código:** master en `bc3ae89`, más dos ramas sin mergear **Apartados:** 11

Cómo leerlo. Los apartados van en el orden en que conviene atacarlos: los de arriba desbloquean a los de abajo. Cada uno lleva una etiqueta de necesidad — IMPRESCINDIBLE no se puede entregar sin ello, IMPORTANTE se nota si falta, SI SOBRA TIEMPO es deuda que no bloquea la entrega.

Ramas. Una rama corta por apartado, salida de `master` y mergeada en cuanto el apartado funcione, igual que se ha hecho con `apartado-de-ventas` o `fotosPerfil`. Así dos personas no se pisan y cada merge deja master arrancable.

El camino crítico es 3 → 4 → 5 → 6: es la cadena que lleva de un colaborador que sube su juego a un cliente que lo compra, recibe su clave y se lo descarga. Es lo que el tercer proyecto necesita que exista de verdad.

Ya fuera de la lista. El apartado de acceso por rol y cierre de sesión se hizo el 6 de noviembre en la rama `seguridad-roles-y-logout`: `/ventas/**` pasó a ADMIN y GESTOR_INVENTARIO, el colaborador aterriza en la tienda en vez de en un 403, el logout dejó de ser un GET que no cerraba nada, y el menú usaba un rol inexistente. Queda **probarlo al arrancar**, que es parte del apartado 1. También está hecho, y nunca figuró aquí, el `UsuarioService` que saca de `ControllerHome` la lógica de la foto de perfil (rama `usuario-service`).

SPRINT 1 · DEJAR EL TERRENO LISTO Y ABRIR EL CIRCUITO

1 Punto de control: recrear la base y arrancar

IMPRESCINDIBLE

> **Hacer `docker compose down -v`, arrancar la aplicación y comprobar que las nueve migraciones se aplican y que Hibernate acepta el esquema.**

V2 se ha editado a mano varias veces, así que la base que haya en los equipos ya no cuadra con la huella que guarda Flyway. Hasta que alguien arranque con la base recreada no sabemos si lo construido desde el 4 de noviembre (imágenes de juego, fotos de perfil, V9) levanta: el compilador no valida ni el esquema ni las plantillas, eso solo sale al arrancar. Es media hora y es la red de seguridad de todo lo demás. Aprovecha para cargar `docs/datosPruebaPedidos.sql` y ver los listados con datos.

De paso se prueban, por primera vez en ejecución, **las dos ramas que están sin mergear**: que un colaborador entra y cae en la tienda, que un gestor de inventario abre Ventas, que un cliente que escriba `/ventas/pedidos` se come un 403, que el botón de cerrar sesión cierra de verdad, y que la foto de perfil se sigue subiendo y borrando bien con el `UsuarioService` nuevo.

Rama recomendada: `punto-control-esquema`

Primero esto. Cualquier apartado de abajo que se empiece sin este paso se construye encima de algo que no sabemos si arranca.

2 Cerrar las decisiones que bloquean código

IMPRESCINDIBLE

> **Sentaros veinte minutos y cerrar las cuatro decisiones abiertas que impiden escribir el apartado 3, y dejarlas escritas en la memoria.**

Son cuatro y las cuatro son baratas de decidir y caras de improvisar a mitad de código: **dónde se guarda el ZIP** de los juegos (carpeta del servidor o almacenamiento aparte; si el hosting borra el disco en cada despliegue, la carpeta no vale y los juegos desaparecen), **si un juego de colaborador nace activo** o lo tiene que activar alguien, **si el colaborador pone precio** o todos sus juegos son gratis, y **si en la tienda conviven** los juegos de distribuidor con los de colaborador. La primera bloquea el apartado 3 entero.

Rama recomendada: `decisiones-pendientes` · solo toca `docs/`

3 El colaborador sube sus juegos

IMPRESCINDIBLE

> **Crear la pantalla donde un colaborador da de alta su juego con sus imágenes y su ZIP, y queda publicado sin que nadie lo revise.**

Es la razón de ser del actor COLABORADOR y hoy no existe ni una línea: no hay paquete `templates/colaborador/`, ni servicio, ni controlador, y `VersionJuego` es una entidad que no usa nadie. Hay que crear `JuegoColaboradorService` y su impl, `ControllerJuegoColaborador`, `VersionJuegoDao`, las plantillas y su CSS, y tocar `SecurityConfig` y el menú lateral. La mitad del trabajo ya está hecha y se reaprovecha: `AlmacenArchivos` guarda y borra archivos con las rutas validadas, y el CRUD de imágenes de videojuego sirve de molde. Lo nuevo de verdad es el ZIP y que el juego se guarde con `colaborador_id` y sin proveedor.

Rama recomendada: `juegos-colaborador`

Depende del apartado 2: hasta decidir dónde vive el ZIP no se puede escribir la parte de la entrega del ejecutable.

4 La tienda del cliente, con datos reales

IMPRESCINDIBLE

> **Convertir `client/tienda.html` en un catálogo de verdad y publicar la ficha del juego en una URL que pueda abrir un cliente.**

Ahora mismo `tienda.html` son doce líneas con la barra de navegación y `ControllerHome.tienda()` devuelve la plantilla sin consultar nada. La ficha (`client/ficha.html`) sí está hecha y bonita, con sus capturas y su precio rebajado, pero solo se sirve como *vista previa* del panel de administración. Toca un listado con sus filtros por categoría, plataforma y precio, el apartado de juegos de colaboradores, y una ruta pública de ficha. El modelo ya da todo lo que hace falta: `precio`, `precio_descuento`, las imágenes por tipo y los N:M de categoría y plataforma.

Rama recomendada: `tienda-cliente`

5 Cesta y pago con Stripe

IMPRESCINDIBLE

> **Montar la cesta del cliente y el pago con Stripe, que al confirmarse convierte la cesta en pedido y factura.**

Es el apartado más grande que queda y el que el profesor pidió expresamente. `Cesta` y `LineaCesta` existen como entidades y no las usa nadie: no hay DAO, ni servicio, ni controlador. Hay que añadir la dependencia de Stripe (todavía no está en el `pom`), crear la sesión de pago, y recoger la confirmación en un *webhook* o en la vuelta del pago para marcar el pedido como PAGADO. **Los datos de tarjeta los teclea el cliente en Stripe, nunca en un formulario nuestro**: la pantalla de pago del prototipo hay que rehacerla con eso en mente. Ojo con dos cosas que ya sabemos: una línea de cesta con cantidad 2 se expande en dos líneas de pedido, porque una línea de pedido es una clave; y la factura tiene que guardar copia del nombre y la dirección, o el día que un cliente se mude le cambiarán las facturas viejas.

Rama recomendada: `cesta-y-pago-stripe`

6 Claves de licencia, biblioteca y descarga

IMPRESCINDIBLE

> **Acuñar la clave al pagar, enseñarla en la biblioteca del cliente y servir el ZIP solo a quien tenga esa clave.**

`ClaveLicencia` es hoy una entidad huérfana: no hay DAO, ni servicio, ni una sola clave en la base. Es la pieza que cierra el circuito. Para un juego de colaborador la clave **nace vendida** al comprar y no cuelga de ninguna línea de compra; para un juego de distribuidor sale de las que entraron por una orden de compra. Falta también declarar la biblioteca en `Cliente` (el `@OneToMany` contra `ClaveLicencia`, el lado dueño ya está) y el controlador de descarga, que antes de servir el ZIP comprueba que la clave es de ese usuario y está VENDIDA. Acuérdate del índice ciego: `codigo_hash` lo rellena el servicio con `IndiceCiego.hash(valor)`, no se calcula solo.

Rama recomendada: `biblioteca-y-claves`

Cuidado con el orden. Sin el apartado 7, los únicos juegos vendibles son los de colaborador y los de stock ilimitado, porque los de distribuidor no tienen claves. Si no da tiempo a hacer compras, la demo se monta con esos y se dice en la memoria.

7 Órdenes de compra y recepción de claves

IMPORTANTE

> **Dar de alta órdenes de compra a un proveedor y recibir las claves que entran por cada línea.**

Es la mitad izquierda del esquema: proveedor → compra → línea de compra → clave. `Compra` y `LineaCompra` existen como entidades, pero solo las menciona `DatosIniciales`: no hay CRUD. El CRUD de proveedores ya está hecho y sirve de punto de partida. Dos reglas que el modelo ya lleva dentro: `cantidad` nula significa ilimitada, y el empleado que firma la compra es obligatorio.

Rama recomendada: `compras-y-recepcion`

Es lo primero que se sacrifica si los apartados 6 y 7 se comen el sprint: sin esto el ERP sigue enseñándose entero por el lado de la venta.

8 Arreglos sueltos del panel

IMPORTANTE

> **Tapar los cuatro agujeros pequeños que un profesor encuentra en dos clics.**

Son baratos y se notan: el formulario **Mis datos** de `clients/configuracionUsuario.html` es un `<form>` sin `action` ni `method` y sin POST en el controlador, así que «Guardar» recarga la página y **pierde los cambios sin avisar** (la foto sí se guarda, eso ya funciona, y el `UsuarioService` nuevo ya es el sitio donde colgar ese guardado; falta decidir qué campos son editables para quien no es cliente); el formulario de edición de colaborador todavía no deja cambiar la contraseña, y hoy no hay ninguna vía para cambiarla, ni el admin ni él mismo; al listado de clientes le falta el botón Limpiar que tienen los otros tres; y `ClienteDao.buscarFiltrado` compara con `like` sin `lower()`, que funciona de milagro porque MySQL ignora las mayúsculas.

Rama recomendada: `arreglos-panel`

9 Dejarlo listo para producción

IMPRESCINDIBLE

> **Sacar el perfil `dev` de `application.properties`, pasar los secretos al entorno y desplegar la aplicación en algún sitio.**

La aplicación tiene que acabar en producción porque los juegos del tercer proyecto se publicarán en ella. Hoy `spring.profiles.active=dev` está fijado en el fichero, así que **un despliegue tal cual cargaría los 34 usuarios de prueba con contraseña conocida, uno de ellos ADMIN**. El perfil tiene que venir del entorno, igual que `LUDUSERP_AES_KEY`, y conviene separar esa clave de la del índice ciego, que ahora es la misma. Y hay que resolver dónde viven las subidas: `app.uploads.dir` apunta a una carpeta relativa al proyecto, que en un hosting que recrea el disco en cada despliegue se lleva por delante las imágenes y los ZIP.

Rama recomendada: `preparar-produccion`

10 Documentación de entrega: PDF, prototipo y memoria

IMPRESCINDIBLE

> **Rehacer el documento de diseño para que cuente lo que de verdad se ha construido, y volver a sincronizar prototipo y memoria.**

El PDF va con más de veinte correcciones pendientes apuntadas en la memoria: sobra la clase Desenvolupador y las tablas de permisos, falta el actor COLABORADOR entero, la jerarquía de Usuari abstracta, los siete roles en vez de cinco, Stripe en lugar del pago simulado, y Categoria y Plataforma como clases. Además promete cosas que no se usan (MapStruct, Bootstrap, nombres en inglés) y dice Spring Boot 3 cuando el pom es 4.1.1. La memoria (`docs/memoriaClaudeByAleix.txt`) se quedó en el 4 de noviembre y no recoge ventas, correo, paginación, el CRUD de videojuegos, las imágenes ni las fotos de perfil. Es trabajo de varias horas y **no se puede dejar para el último día**, porque es la mitad de lo que se corrige.

Rama recomendada: `documentacion-entrega`

11 Cuatro pruebas y las chapuzas conocidas

SI SOBRA TIEMPO

> **Escribir al menos un test que arranque el contexto, y barrer la deuda pequeña que está apuntada desde octubre.**

No hay ni un test y `src/test` ni siquiera existe, mientras el pom declara ocho *starters* de prueba. Un solo test de carga de contexto habría pillado el choque de entidades del 29 de octubre y el de la columna `activo` del 4 de noviembre. Con eso y cuatro pruebas de servicio ya se justifica el apartado. De paso: ningún servicio lleva `@Transactional` salvo el de clientes, así que validar y guardar no es atómico; no hay `mvnw`; el *healthcheck* de MySQL de `compose.yaml` debería ser CMD-SHELL; `nbactions.xml` arrastra `-noverify`, obsoleta desde JDK 13; y el pom sigue con `artifactId demo` y `groupId com.example`.

Rama recomendada: `tests-minimos`

RESUMEN

#	APARTADO	RAMA	SPRINT	NECESIDAD
1	Punto de control: recrear la base y arrancar	`punto-control-esquema`	1, día 1	Imprescindible
2	Cerrar las decisiones abiertas	`decisiones-pendientes`	1, día 1	Imprescindible
3	El colaborador sube sus juegos	`juegos-colaborador`	1	Imprescindible
4	La tienda del cliente, con datos reales	`tienda-cliente`	1	Imprescindible
5	Cesta y pago con Stripe	`cesta-y-pago-stripe`	2	Imprescindible
6	Claves, biblioteca y descarga	`biblioteca-y-claves`	2	Imprescindible
7	Órdenes de compra y recepción	`compras-y-recepcion`	2	Importante
8	Arreglos sueltos del panel	`arreglos-panel`	2	Importante
9	Dejarlo listo para producción	`preparar-produccion`	2	Imprescindible
10	Documentación de entrega	`documentacion-entrega`	1 y 2, a ratos	Imprescindible
11	Pruebas mínimas y deuda pequeña	`tests-minimos`	2	Si sobra tiempo
—	Acceso por rol y cierre de sesión · **hecho el 6 de noviembre**, pendiente de probar al arrancar	`seguridad-roles-y-logout`	—	Hecho
—	UsuarioService: la foto de perfil sale de ControllerHome · **hecho el 7 de noviembre**	`usuario-service`	—	Hecho

Si el tiempo se tuerce, el orden de recorte es: primero el 11, después el 7, y luego la mitad opcional del 8. Lo que no se puede recortar es la cadena 3 → 4 → 5 → 6, ni el 9 ni el 10: sin ellos no hay ni aplicación en producción ni documento que entregar.